

Abstract Structure

1. Introduction

This document provides a summary of the final implementation of a relational database system for an Airbnb-style property rental platform. The project was organized into three phases: conception (ER modeling), development (SQL implementation), and finalization (refinement and documentation). The database accommodates four user roles—Guest, Host, Admin, and Support—and efficiently manages properties, bookings, payments, reviews, and communications. All data has been normalized to the third normal form, and referential integrity is upheld using primary and foreign keys.

2. Database Functionality

The database supports an online booking platform with the following features:

- **User Management:** User registration, role-based permissions, profile verification, and social media links.
- **Property Management:** Hosts can list properties with descriptions, amenities, location, pricing, and availability.
- **Booking Engine:** Guests can book properties with automatic pricing, availability checks, and status tracking (pending, confirmed, completed, canceled).
- **Payment Processing:** Secure payments through various methods (credit card, PayPal) with status tracking.
- **Review System:** Guests and hosts can rate each other after stays, with reviews linked to bookings.
- **Communication:** Messaging between users related to specific bookings.
- **Dispute Resolution:** Administrators manage disputes between guests or hosts.
- **Audit Logging:** Critical actions are logged for security and troubleshooting.
- **Reporting:** Dashboards provide insights into bookings, revenue, and ratings.

Advanced features implemented at the database level include:

- **Triple relationships:** (1) Booking connects Guest, Property, and BookingStatus; (2) Payment connects Booking, PaymentMethod, and PaymentStatus; (3) Review connects Booking, Reviewer, and Reviewee.
- **Recursive relationships:** (1) Message between users; (2) Review between users (self-reference).
- **Stored procedures:** `sp_make_booking` and `sp_process_payment` encapsulate business logic.
- **Triggers:** Prevent double-booking, automatically update timestamps, and maintain host reports.
- **Functions:** `fn_calculate_property_rating` and `fn_check_availability` provide reusable calculations.

3. Metadata

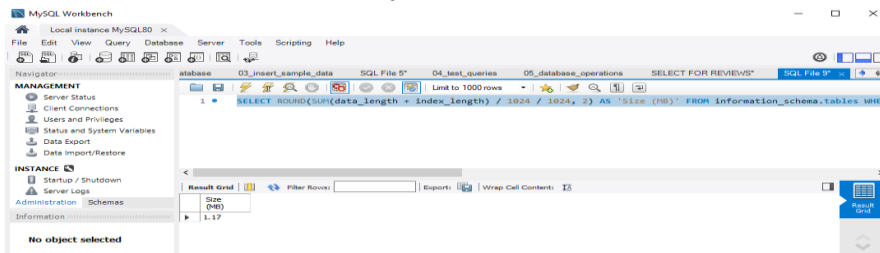
The final database contains the following objects and data volumes:

Object Type	Name / Count
Tables	22
Views	3 (vw_property_listings, vw_booking_details, vw_host_dashboard)
Stored Procedures	2 (sp_make_booking, sp_process_payment)
Function	2 (fn_calculate_property_rating, fn_check_availability)
Triggers	3 (trg_booking_updated, trg_prevent_double_booking, trg_update_host_report)
Indexes	12 (including those created with tables and later added)

Sample Data Counts:

TABLE	ROW COUNT
Role	4
User	24
Property	22
Booking	22
Payment	22
Review	14
Message	12
Amenity	28
Location	23
PropertyType	13
PropertyAmenity	47
BookingStatus	6
PaymentMethod	6
Payout	22
AvailabilityCalendar	24
Dispute	3
Auditing	5
SocialLink	5
HostReport	5
CancellationPolicy	5
Verification	5

Database Size: Approximately [1.17] MB



4. Installation Guide

To deploy the database on any MySQL system (version 8.0+ recommended), follow these steps:

1. **Create the database**

Execute `01_create_database.sql` to create `airbnb_db`.

2. **Create tables**

Run `02_create_tables.sql` to build all 22 tables with constraints and indexes.

3. **Populate sample data**

Execute `03_insert_sample_data.sql`. The insert order respects foreign key dependencies, so no errors should occur.

4. **Test the implementation**

Run `04_test_queries.sql` to verify triple and recursive relationships, and to see example business queries.

5. **Create advanced objects**

Execute `05_database_operations.sql` to add views, stored procedures, triggers, and functions.

All SQL files are provided in the submission ZIP. The database can be managed using any MySQL client (e.g., MySQL Workbench, command line).

1. Clone the repository:

```
git clone https://github.com/ipadeolaoluwatoyin7880/
```

```
airbnb-sql-data-mart.git
```

2. Execute scripts in order:

```
mysql -u root -p < 01_create_database.sql
```

```
mysql -u root -p airbnb_db < 02_create_tables.sql
```

```
mysql -u root -p airbnb_db < 03_insert_sample_data.sql
```

```
mysql -u root -p airbnb_db < 04_test_queries.sql
```

```
mysql -u root -p airbnb_db < 05_database_operations.sql
```

5. Conclusion and Lessons Learned

This project successfully delivered a normalized database for an Airbnb-like platform. Key lessons include:

- Future enhancements will include full-text search for property descriptions, geospatial queries, and effective partitioning to ensure better scalability.

